Nocions bàsiques SQL

Nivell 1

Exercici 1

A partir dels documents adjunts (estructura_dades i dades_introduir), importa les dues taules. Mostra les característiques principals de l'esquema creat i explica les diferents taules i variables que existeixen. Assegura't d'incloure un diagrama que il·lustri la relació entre les diferents taules i variables.

Respuesta:
En el primer archivo se crea la BBDD Transactions y también 2 tablas (transaction y company). El segundo archivo se carga los datos en las tablas previamente creadas. Con relación a las características el **modelo de relación entre las tablas es 1 - N (**1 empresa tiene N transacciones y cada transacción solo corresponde a una empresa)

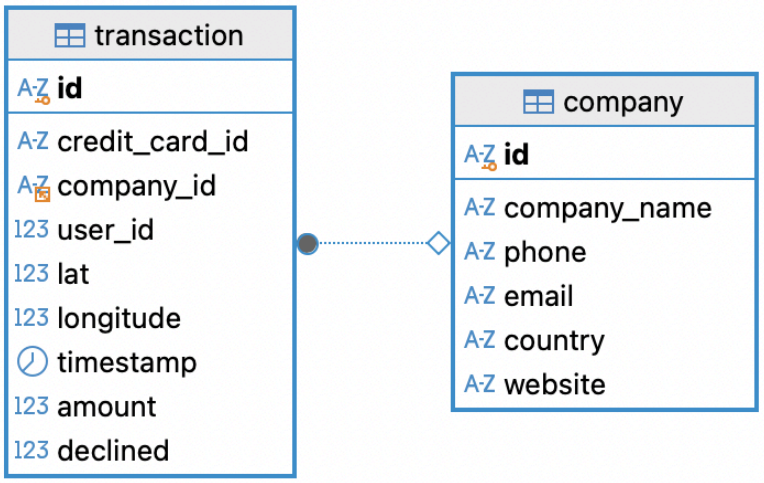


Tabla Company:

Todas las columnas son campos de texto, también todas (menos la PK que es id) tienen registros con nulos:

Field	Type	Null	Key	Default
id	varchar(15)	NO	PRI	NULL
company_name	varchar(255)	YES		NULL
phone	varchar(15)	YES		NULL
email	varchar(100)	YES		NULL
country	varchar(100)	YES		NULL
website	varchar(255)	YES		NULL

Tabla Transaction:

Igualmente, todas las columnas tienen registros nulos (menos la PK id), tenemos una FK que conecta con la tabla anterior company_id.
Aquí los campos son diversos, tenemos los textos (varchar), enteros (int), decimales (float y decimal), tiempo (timestamp - fecha y hora)

Field	Type	Null	Key	Default
id	varchar(255)	NO	PRI	NULL
credit_card_id	varchar(15)	YES		NULL
company_id	varchar(20)	YES	MUL	NULL
user_id	int	YES		NULL
lat	float	YES		NULL
longitude	float	YES		NULL
timestamp	timestamp	YES		NULL
amount	decimal(10,2)	YES		NULL
declined	tinyint(1)	YES		NULL

Por lo que veo en el código más adelante se deberán crear 2 nuevas tablas: usuarios y tarjetas de crédito más que nada por las líneas:

- `credit_card_id VARCHAR(15) REFERENCES credit_card(id)`
- `user_id INT REFERENCES user(id),`

Exercici 2

Utilitzant JOIN realitzaràs les següents consultes:

- **Llistat dels països que estan generant vendes.**

```
9   -- Exercici 2
10  SELECT DISTINCT company.country AS Países
11  FROM transaction AS t
12  JOIN company ON t.company_id = company.id
13  WHERE t.declined = false
14  ORDER BY company.country;
15
```

Output | Result #2 | Result #3 | Result #4 | Result #5

Países
Australia
Belgium
Canada
China
France
Germany
Ireland
Italy
Netherlands
New Zealand
Norway
Spain
Sweden
United Kingdom
United States

- **Des de quants països es generen les vendes.**

```
16  SELECT COUNT(DISTINCT company.country) AS Cantidad_Países_Venta
17  FROM transaction AS t
18  JOIN company ON t.company_id = company.id
19  WHERE t.declined = false;
```

Output | Result #2 | Result #3 | Result #4 | Result #5 | Result #6 | Res

Cantidad_Países_Venta
15

- **Identifica la companyia amb la mitjana més gran de vendes.**

```
21
22      SELECT company.company_name AS Empresa,
23      AVG(t.amount) AS Top_Media_Ventas
24      FROM transaction AS t
25      JOIN company ON t.company_id = company.id
26      WHERE t.declined = false
27      GROUP BY Empresa
28      ORDER BY Top_Media_Ventas DESC
29      LIMIT 1;
```

Output | Result #1 | Result #2 | Result #3 | Result #4

Empresa	Top_Media_Ventas
Ac Fermentum Incorporated	284.911333

Exercici 3

Utilitzant només subconsultes (sense utilitzar JOIN):

- **Mostra totes les transaccions realitzades per empreses d'Alemanya.**

```
33  • SELECT *
34    FROM transaction AS t
35    WHERE t.company_id IN (SELECT c.id
36                           FROM company AS c
37                           WHERE c.country = 'Germany');
```

Output | Result #2 | Result #3 | Result #4 | Result #5 | Result #6 | Result #7 | Result #8 | Result #9 | Result #10

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
00138D3B-206D-4C03-94B7-63A2676EB9B4	CcS-4899	b-2222	318	41.3781	12.447	03/25/2020	426.36	false
0013C1B6-3B84-4D6C-8154-E2B3FEBCA8E9	CcS-5070	b-2222	489	41.3814	2.18176	12/17/2020	316.90	false
00201A11-2E62-44C4-941D-198FC8DB77F0	CcU-3512	b-2222	193	55.5704	-3.65129	01/22/2021	453.04	false
00235618-0A5C-4D49-9DCB-B3A9405D8923	CcS-8137	b-2222	3556	59.8421	18.729	09/09/2020	263.14	false

- **Llista les empreses que han realitzat transaccions per un amount superior a la mitjana de totes les transaccions.**

```sql
SELECT c.company_name AS Empresa
FROM company AS c
WHERE c.id IN (
    SELECT company_id
    FROM transaction
    WHERE amount > (SELECT AVG(amount)
                    FROM transaction));
```

Output | Result #1

Empresa
Ac Fermentum Incorporated
Magna A Neque Industries
Fusce Corp.
Convallis In Incorporated
Ante Iaculis Nec Foundation
Donec Ltd
Sed Nunc Ltd
Amet Nulla Donec Corporation
Nascetur Ridiculus Mus Inc.
Vestibulum Lorem PC
Gravida Sagittis LLP
Mus Aenean Eget Foundation
Dis Parturient Institute
Sed LLC
Arcu LLP
Pretium Neque Corp.
Fringilla LLC
Quisque Libero LLC
Auctor Mauris Vel LLP
Elit Etiam Laoreet Associates
Nunc Interdum Incorporated
Augue Foundation
Non Magna LLC
A Institute
Quam A Felis Industries
Integer Mollis Corp.
Enim Condimentum Ltd
Donec Fringilla PC
Amet Institute
Magna Incorporated
Fringilla Porttitor Incorporated
Ut Semper Foundation

- **Eliminaran del sistema les empreses que no tenen transaccions registrades, entrega el llistat d'aquestes empreses.**

```sql
SELECT c.company_name AS Empresa_sin_transacciones
FROM company AS c
WHERE c.id NOT IN (
    SELECT company_id
    FROM transaction
    WHERE company_id IS NOT NULL);
-- Output a 0 comprobación
SELECT COUNT(*) FROM company;
SELECT COUNT(DISTINCT company_id) FROM transaction;
-- en ambas consultas aparecen la misma cantidad de empresas por lo que no hay nada para eliminar
```

Output

Exercici 4

La teva tasca és dissenyar i crear una taula anomenada "credit_card" que emmagatzemi detalls crucials sobre les targetes de crèdit. La nova taula ha de ser capaç d'identificar de manera única cada targeta i establir una relació adequada amb les altres dues taules ("transaction" i "company"). Després de crear la taula serà necessari que ingressis la informació del document denominat "dades_introduir_credit". Recorda mostrar el diagrama i realitzar una breu descripció d'aquest.

Al crear la tabla, establezco ID como PK porque se ha comprobado que es una columna con valores únicos; luego pongo todas las variables como texto para facilitar la carga y, si más adelante necesito hacer algún cálculo o filtrar por fecha, ya se hará un alter table.
La relación entre las tablas se hace manteniendo transaction como tabla de hechos y credit_card como dimensión.

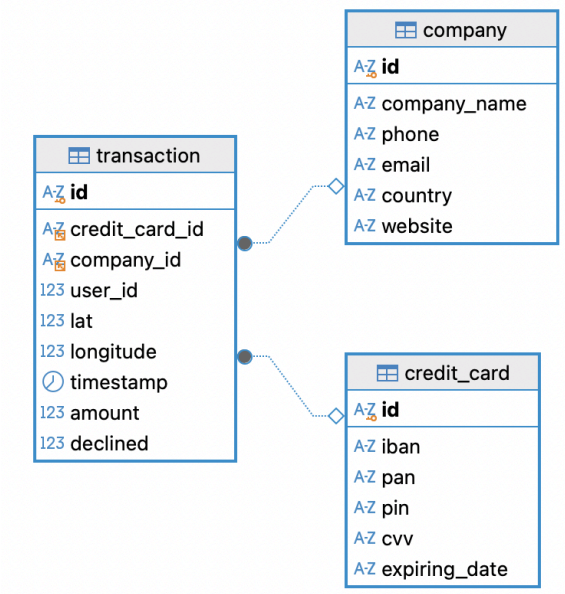
```sql
-- Exercici 4
CREATE TABLE IF NOT EXISTS credit_card (
    id VARCHAR(15) PRIMARY KEY,
    iban VARCHAR(34),
    pan VARCHAR(20),
    pin VARCHAR(4),
    cvv VARCHAR(3),
    expiring_date VARCHAR(10)
);
-- (Comprobrando carga de datos) Mirar si en la tabla transaction tiene alguna tarjeta que no está en la tabla credict_card
SELECT DISTINCT credit_card_id
FROM transaction
WHERE credit_card_id NOT IN (SELECT id FROM credit_card);
-- Crear relacion entre tablas
ALTER TABLE transaction
ADD CONSTRAINT fk_transaction_credit_card
FOREIGN KEY (credit_card_id) REFERENCES credit_card(id);

```

Output

```
OK, 100000 rows affected in 3.8137826919555664s
```

*** Lo del CREATE TABLE IF NOT EXISTS es solo para poder rodar el archivo varias veces sin error.**



El nuevo diagrama muestra la nueva tabla credict_card y la relación con la tabla principal transaction, que es de la misma naturaleza que la relación de las tablas anteriores (1:N) 1 tarjeta de crédito puede tener varias transacciones, pero una transacción solo tiene 1 tarjeta de crédito.

Exercici 5

El departament de Recursos Humans ha identificat un error en el número de compte associat a la targeta de crèdit amb ID CcU-2938. La informació que ha de mostrar-se per a aquest registre és: TR323456312213576817699999. Recorda mostrar que el canvi es va realitzar.

```
75
76    -- Exercici 5 - primero hago la consulta para localizar el registro:
77    SELECT *
78    FROM credit_card
79    WHERE id = 'CcU-2938';
80
```

Output | Result #1

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR301950312213576817638661	5424465566813633	3257	984	10/30/22

Una vez localizado el registro procedo la actualización del campo IBAN que es lo que pide RRHH.

```
80    -- Actualización de IBAN tal como pide RRHH
81    UPDATE credit_card
82    SET iban = 'TR323456312213576817699999'
83    WHERE id = 'CcU-2938';
84
```

Output

```
OK, 1 row affected in 26.438ms
```

Vi que el cambio solo se aplicó en una línea, lo que es correcto, ahora hago otra vez la primera consulta y compruebo el cambio en la BBDD.

```
76    -- Exercici 5 - primero hago la consulta para localizar el registro:
77    SELECT *
78    FROM credit_card
79    WHERE id = 'CcU-2938';
80    -- Actualización de IBAN tal como pide RRHH
81    UPDATE credit_card
82    SET iban = 'TR323456312213576817699999'
83    WHERE id = 'CcU-2938';
84    -- Comprobación de cambio en la BBDD
85    SELECT *
86    FROM credit_card
87    WHERE id = 'CcU-2938';
```

Output | Result #1

id	iban	pan	pin	cvv	expiring_date
CcU-2938	TR323456312213576817699999	5424465566813633	3257	984	10/30/22

Exercici 6

En la taula "transaction" ingressa una nova transacció amb la següent informació:

Id 108B1D1D-5B23-A76C-55EF-C568E49A99DD // credit_card_id CcU-9999 // company_id b-9999 // user_id 9999 // lat 829.999 // longitude -117.999 // amount 111.11 // declined 0

En las columnas ofrecidas no sale la columna timestamp y he decidido no ponerla en mi código para que se rellene con lo que es default de la tabla.
Por otro lado, antes de insertar la nueva línea, he verificado en las otras tablas (company y credict_card) si existía la información para que no me truncara la inserción de los datos, y efectivamente no existía, por lo que primero incluyo la información que falta en las tablas de dimensiones y luego inserto la línea tal como pide el ejercicio.

```
89    -- Exercici 6
90    -- comprobación de entradas
91    SELECT * FROM credit_card WHERE id = 'CcU-9999';
92    SELECT * FROM company WHERE id = 'b-9999';
93    -- Inserción de registros en las tablas:
94    INSERT INTO credit_card (id) VALUES ('CcU-9999');
95    INSERT INTO company (id) VALUES ('b-9999');
96    -- Inserción de linea del enunciado sin timestamp, respetando la categoria de cada campo
97    INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
98    VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', 9999, 829.999, -117.999, 111.11, 0);
99
100
```

Output

```
OK, 1 row affected in 2.434ms
```

Por fin solamente compruebo si el registro de la nueva linea se ve correctamente en la tabla.

```
89    -- Exercici 6
90    -- comprobación de entradas
91    SELECT * FROM credit_card WHERE id = 'CcU-9999';
92    SELECT * FROM company WHERE id = 'b-9999';
93    -- Inserción de registros en las tablas:
94    INSERT INTO credit_card (id) VALUES ('CcU-9999');
95    INSERT INTO company (id) VALUES ('b-9999');
96    -- Inserción de linea del enunciado sin timestamp, respetando la categoria de cada campo
97    INSERT INTO transaction (id, credit_card_id, company_id, user_id, lat, longitude, amount, declined)
98    VALUES ('108B1D1D-5B23-A76C-55EF-C568E49A99DD', 'CcU-9999', 'b-9999', 9999, 829.999, -117.999, 111.11, 0);
99    -- Comprobación de la inserción de lina con informaciones descritas
100   SELECT * FROM transaction
101   WHERE id = '108B1D1D-5B23-A76C-55EF-C568E49A99DD';
102
```

Output | Result #1

id	credit_card_id	company_id	user_id	lat	longitude	timestamp	amount	declined
108B1D1D-5B23-A76C-55EF-C568E49A99DD	CcU-9999	b-9999	9999	829.999	-117.999	NULL	111.11	false

Exercici 7

Des de recursos humans et sol·liciten eliminar la columna "pan" de la taula credit_card. Recorda mostrar el canvi realitzat.

Primero hago un SHOW COLUMNS para verificar todas las columnas de la tabla, en una segunda etapa borro la columna pan y por fin otra vez el paso inicial para verificar el cambio.

```
103    -- Exercici 7
104    -- Para registrar el antes y despues del ejercicico:
105    SHOW COLUMNS FROM credit_card;
106    -- Para borrar la columna
107    ALTER TABLE credit_card
108    DROP COLUMN pan;
109    -- Para verificar el cambio
110    SHOW COLUMNS FROM credit_card;
```

Output | Result #1

Field	Type	Null	Key	Default	Extra
id	varchar(15)	NO	PRI	NULL	
iban	varchar(34)	YES		NULL	
pan	varchar(20)	YES		NULL	
pin	varchar(4)	YES		NULL	
cvv	varchar(3)	YES		NULL	
expiring_date	varchar(10)	YES		NULL	

```
103    -- Exercici 7
104    -- Para registrar el antes y despues del ejercicico:
105    SHOW COLUMNS FROM credit_card;
106    -- Para borrar la columna
107    ALTER TABLE credit_card
108    DROP COLUMN pan;
109    -- Para verificar el cambio
110    SHOW COLUMNS FROM credit_card;
```

Output | Result #1

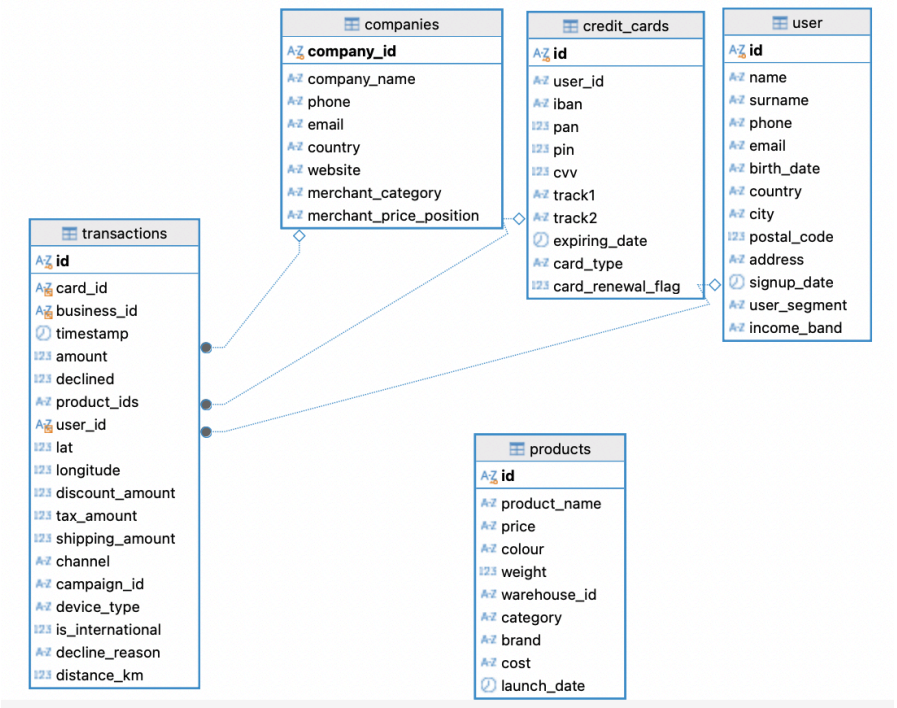
Field	Type	Null	Key	Default	Extra
id	varchar(15)	NO	PRI	NULL	
iban	varchar(34)	YES		NULL	
pin	varchar(4)	YES		NULL	
cvv	varchar(3)	YES		NULL	
expiring_date	varchar(10)	YES		NULL	

Exercici 8

Estudia'ls i dissenya una base de dades amb un esquema d'estrella que contingui, almenys 4 taules de les quals puguis realitzar les següents consultes:
La taula de products.csv l'utilitzarem més endavant.

Primero he creado un archivo a parte de la creacion de la BBDD y carga de informacion de las tablas. Luego vi que 2 archivos hacen referencia a usuarios (europeos y americanos) por lo que he decidido crear la tabla users y subir los 2 archivos en la misma tabla.

- Todas las relaciones establecidas son de 1:N
- La tabla de hechos sigue siendo la de transactions y las otras son las dimensiones, de momento un modelo estrella.



Exercici 9

Realitza una subconsulta que mostri tots els usuaris amb més de 80 transaccions utilitzant almenys 2 taules.

La consulta de afuera tiene que ser sobre la tabla de usuarios ya que es esto que pide la pregunta, la consulta de dentro se hace con la tabla de transactions por el conteo de transacciones, filtro con la segunda consulta contando los usuarios con más de 80 transacciones y agrupando por id de usuario.

```
265
266     -- Exercici 9
267   • SELECT *
268     FROM user
269     WHERE id IN (SELECT user_id
270                  FROM transactions
271                  GROUP BY user_id
272                  HAVING COUNT(id) > 80);
273
274     -- Exercici 10
```

Ln 272, Col 37

Output | Result #1

id	name	surname	phone	email	birth_date	country	city
185	Molly	Gilliam	0800 120 8023	donec@outlook.couk	Dec 21, 1993	United Kingdom	London
289	Dxwgi	Hwcru	+98-309-8797	dxwgi.hwcru@example.com	Aug 20, 1976	Germany	Stuttgart
318	Bnyr	Astuw	+33-120-9644	bnyr.astuw@example.com	May 3, 1974	Italy	Genoa
454	Sfzzoh	Xgvfridxs	+58-495-3945	sfzzoh.xgvfridxs@example.com	Aug 28, 1962	Poland	Gdansk

Exercici 10 -

Mostra la mitjana d'amount per IBAN de les targetes de crèdit a la companyia Donec Ltd, utilitza almenys 2 taules.

Primero tenemos que filtrar la empresa “DONEC” en la tabla de companies y selecciono el id de la empresa porque es la conexión con la otra tabla que es la de transacciones, como entiendo que las ventas son las efectivadas añado la condicion declined = false, encaso que se quiera el total de transacciones entonces se puede comentar esta parte.

En la consulta de afuera, entonces selecciono el iban que está en la tabla credit_card, la media de ventas que está en la tabla transaction (aquí ya tengo por fuerza que hacer un join), luego filtro por el id de empresa que es la conexión con la subconsulta.

```
274  -- Exercici 10
275  SELECT cc.iban, ROUND(AVG(t.amount), 2) AS Media_Amount
276  FROM transactions AS t
277  JOIN credit_cards AS cc ON cc.id = t.card_id
278  WHERE t.business_id = (SELECT company_id
279                         FROM companies
280                         WHERE company_name = 'Donec Ltd') AND t.declined = 0
281  GROUP BY cc.iban;
282
```

Output | Result #1

iban	Media_Amount
CA90135577358677162610899O	361.01
US72921264281695565142879l	142.56
US89945080320933276819251	246.90
US58109743266832554114229 7	154.01
CA44122626710585747357057 4	250.51
CA33217196235207301508677 5	203.95
US98535962870119888062872O	438.16
US78811197299769367552491 4	520.37
CA86788652925857744204989 8	204.63
US45095074240038802316200 2	165.08
US84158923479572285734864 6	253.93
CA70735050385393396856348 7	290.57
US22517429614700226563119O	240.32
CA81832525035226964072008 8	86.52
US57008242928690160059186 6	288.19
US18339162660537723091668 9	83.54
US35961752242517862939184 1	96.81
US49486568270958047955270 3	158.25

OK, 367 records retrieved in 82.577ms

Nivel 2 - Exercici 1

Identifica els cinc dies que es va generar la quantitat més gran d'ingressos a l'empresa per vendes. Mostra la data de cada transacció juntament amb el total de les vendes.

Esta consulta de la tabla transaction, desde la columna timestamp, tengo que sacar solamente la fecha y agruparlas, sumar las ventas, filtrar por las que no fueron efectivas (por el declined) y luego ordenar descendente, para que aparezca en el output desde el dia que ha tenido mas ventas para el que menos, por fin limitar a 5.

```
283    -- Nivel 2 - Exercici 1
284    SELECT DATE(t.timestamp) AS Fecha,
285           ROUND(SUM(t.amount), 2) AS Total_Ventas
286    FROM transactions AS t
287    WHERE declined = 0
288    GROUP BY Fecha
289    ORDER BY Total_Ventas DESC
290    LIMIT 5;
```

Output | Result #1

Fecha	Total_Ventas
02/20/2023	15728.40
11/18/2019	15447.89
12/13/2022	14944.00
03/18/2019	14896.47
12/20/2017	14367.96

OK, 5 records retrieved in 301.992ms

Nivel 2 - Exercici 2

Presenta el nom, telèfon, país, data i amount, d'aquelles empreses que van realitzar transaccions amb un valor comprès entre 350 i 400 euros i en alguna d'aquestes dates: 29 d'abril del 2015, 20 de juliol del 2018 i 13 de març del 2024. Ordena els resultats de major a menor quantitat.

El nombre, telefono y pais de la empresa (tabla company), fecha y amount (tabla transaction) ya veo el join por el output que me pide, filtrar por los valores de las transacciones between, otro filtro por las fechas, otro de las ventas efectivas (declined), tenemos que hacer lo mismo que en la consulta anterior sacar DATE de timestamp por fin ordenar amount de manera descendiente.

```
292     -- Nivel 2 - Exercici 2
293     SELECT c.company_name AS Nombre,
294            c.phone AS Telefono,
295            c.country AS Pais,
296            DATE(t.timestamp) AS Fecha,
297            t.amount AS Amount
298     FROM transactions AS t
```

Output | Result #1

Nombre	Telefono	Pais	Fecha	Amount
Fringilla LLC	+1 150 784 1716	New Zealand	04/29/2015	390.33
Auctor Mauris Vel LLP	+32 133 447 823	United States	03/13/2024	383.58
Mauris Institute	+75 473 619 947	Sweden	04/29/2015	369.20
Pede Cum Ltd	+26 625 798 612	Norway	07/20/2018	358.64
Netus Et Malesuada Ltd	+52 818 402 163	Netherlands	03/13/2024	352.21
Tempor Diam Institute	+89 848 141 313	Netherlands	03/13/2024	350.71

OK, 6 records retrieved in 84.295ms

Nivel 2 - Exercici 3

Necessitem optimitzar l'assignació dels recursos i dependrà de la capacitat operativa que es requereixi, per la qual cosa et demanen la informació sobre la quantitat de transaccions que realitzen les empreses, però el departament de recursos humans és

exigent i vol un llistat de les empreses on especifiquis si tenen igual o més de 400 transaccions o menys.

Lo que pide es la cantidad transaccciones por empresa (tabla transaction, groupby id.empresa,) pero para sacar el nombre de la empresa hace falta un join con company, y si está en la consulta también tiene que estar en groupby. Luego hacer una clasificacion entre las >= 400 y < 400 y ordenar decreciente para poder mirar si la clasificacion fue correcta.

```sql
-- Nivel 2 - Exercici 3
SELECT c.company_name AS Empresa,
       COUNT(t.id) AS Num_Transacciones,
       CASE
           WHEN COUNT(t.id) >= 400 THEN '400 o más'
           ELSE 'Menos de 400'
       END AS Clasificacion
FROM transactions AS t
JOIN companies AS c ON c.company_id = t.business_id
WHERE t.declined = 0
GROUP BY c.company_id, Empresa
ORDER BY Num_Transacciones DESC; -- simplemente para conferir si la clasificación fue correcta
```

Output | Result #1

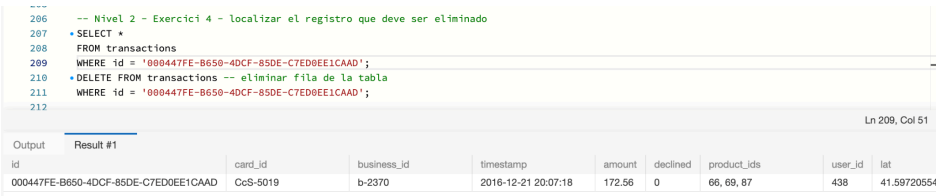
Empresa	Num_Transacciones	Clasificacion
Ac Fermentum Incorporated	2387	400 o más
Nunc Interdum Incorporated	1582	400 o más
Donec Fringilla PC	1578	400 o más
Mauris Institute	1575	400 o más
Rutrum Non Inc.	1573	400 o más
Aliquet Vel Vulputate Incorporated	1570	400 o más
Etiam Bibendum Fermentum Industries	1558	400 o más
Aliquam PC	1554	400 o más
Auctor Mauris Corp.	1554	400 o más
Mus Aenean Eget Foundation	1547	400 o más
Orci Adipiscing Limited	1547	400 o más
Enim Condimentum Ltd	1540	400 o más

OK, 100 records retrieved in 294.386ms

Nivel 2 - Exercici 4

Elimina de la taula transaction el registre amb ID 000447FE-B650-4DCF-85DE-C7ED0EE1CAAD de la base de dades.

Hacer un select para buscar el registro, eliminar la fila y otra vez hacer el select del registro para comprobar que sale vacio.







Nivel 2 - Exercici 5

La secció de màrqueting desitja tenir accés a informació específica per a realitzar anàlisi i estratègies efectives. S'ha sol·licitat crear una vista que proporcioni detalls clau sobre les companyies i les seves transaccions. Serà necessària que creïs una vista anomenada VistaMarketing que contingui la següent informació: Nom de la companyia. Telèfon de contacte. País de residència. Mitjana de compra realitzat per cada companyia. Presenta la vista creada, ordenant les dades de major a menor mitjana de compra.

La cuestión pide nombre empresa, teléfono, país de residencia (tabla company), mean de compra y venta efectiva (tabla transaction), aquí ya tenemos join company + transaction), group by company.id y todo lo que está en select, orderby descending

```sql
-- Nivel 2 - Exercici 5
CREATE VIEW Vista_Marketing AS
SELECT c.company_name AS Nombre_Empresa,
       c.phone AS Telefono,
       c.country AS Pais,
       ROUND(AVG(t.amount), 2) AS Media_Compra
FROM transactions AS t
JOIN companies AS c ON c.company_id = t.business_id
WHERE t.declined = 0
GROUP BY c.company_id, Nombre_Empresa, Telefono, Pais;
```

Output

```
OK, 0 records retrieved in 10.418ms
```

```sql
-- Nivel 2 - Exercici 5
CREATE VIEW Vista_Marketing AS
SELECT c.company_name AS Nombre_Empresa,
       c.phone AS Telefono,
       c.country AS Pais,
       ROUND(AVG(t.amount), 2) AS Media_Compra
FROM transactions AS t
JOIN companies AS c ON c.company_id = t.business_id
WHERE t.declined = 0
GROUP BY c.company_id, Nombre_Empresa, Telefono, Pais;

SELECT *
FROM Vista_Marketing
ORDER BY Media_Compra DESC;
```

Output | Result #1

Nombre_Empresa	Telefono	Pais	Media_Compra
Ac Fermentum Incorporated	+64 601 244 632	Germany	330.71
Ut Semper Foundation	+10 283 500 644	Sweden	307.30
Dolor Vitae Limited	+6 421 567 9195	France	306.21
Orci Adipiscing Limited	+60 897 390 582	United Kingdom	305.87
Nunc In Foundation	+13 267 741 261	Italy	305.67
Maecenas Malesuada Fringilla Inc.	+51 673 540 528	Netherlands	304.49
Turpis Company	+17 624 311 716	Netherlands	303.47
Nulla Integer Vulputate Corp.	+75 236 687 916	Sweden	303.28
Non Justo Corp.	+67 963 294 166	Sweden	303.17
Aliquet Sem Limited	+44 744 425 991	Netherlands	302.77
Aliquam Erat Volutpat LLP	+39 372 330 722	Italy	302.24
Placerat LLP	+17 324 602 503	Netherlands	302.19
Amet Lorem LLP	+40 468 847 773	Spain	301.11
Integer Mollis Corp.	+38 803 863 788	Italy	300.79

OK, 100 records retrieved in 368.725ms

Nivel 3 - Exercici 1

Crea una nova taula que reflecteixi l'estat de les targetes de crèdit basat en si les tres últimes transaccions han estat declinades aleshores és inactiu, si almenys una no és rebutjada aleshores és actiu. Partint d'aquesta taula respon:

👉 **Quantes targetes estan actives?**

Para saber si las transacciones han sido declinadas tengo que buscar en la columna declined (BOLEANO > 0 = false / 1 = verdadero). Si son las 3 ultimas declinadas y cada declinada vale 1 si la suma es > = 3 se estrablece como inactiva

```sql
-- Nivel 3 - Exercici 1
CREATE TABLE IF NOT EXISTS estado_tarjeta AS
SELECT card_id,
    CASE
        WHEN SUM(declined) = 3 THEN 'inactiva'
        ELSE 'activa'
    END AS status
FROM (
    SELECT card_id, declined,
        ROW_NUMBER() OVER (PARTITION BY card_id ORDER BY timestamp DESC) AS rn
    FROM transactions
) AS ranking
WHERE rn <= 3
GROUP BY card_id;

-- Por fin la consulta solicitada
SELECT COUNT(*) AS Tarjetas_Activas
FROM estado_tarjeta
WHERE status = 'activa';
```

Output | Result #1

Tarjetas_Activas
5000

Nivel 3 - Exercici 2

Crea una taula amb la qual puguem unir les dades de l'arxiu de products.csv amb la base de dades creada (ja que fins ara no podíem fer-ho), tenint en compte que des de transaction tens product_ids. Genera la següent consulta:

👉 **Necessitem conèixer el nombre de vegades que s'ha venut cada producte.**

El problema principal de la unión de las tablas es que es imposible tener una FK en products que conecte unicamente con una linea de la tabla de productos ya que la columna product_ids hace referencia a varios productos, es decir, que en una transacción se compraron distintos SKUs.

```sql
SELECT id, product_ids
FROM transactions
LIMIT 10;
```

Output | Result #1

id	product_ids
CDDA7E40-544D-47BB-A4ED-671DD8A950D9	75, 73, 98
09456357-8E9B-475A-8257-87A023699964	3
C47C7C84-C174-4973-A76B-825A9DB5582A	92, 85, 36, 23
2FB526AA-3844-4DDF-AC09-2EBCC255EE76	5, 27
0D877407-B85E-40D2-9229-6BD6F2B8DF0A	38, 32, 82, 62
A0BD09EE-7199-41CF-9CB5-3205509FA02F	92, 64
29322BF7-84F4-4A6C-9FFC-C60B6BBD8DCD	6, 12, 100, 22, 23
3F614E9B-1EEC-4FB7-8B88-ABCD6C519EED	69, 97, 31
133534F1-81E5-4688-B4AF-54E516504E9E	51, 3, 68
185D9EC6-9540-4E4F-B38D-395B26C467AC	69

Como la relación entre las tablas product y transaction era de N:M (una transacción puede incluir **varios productos** y un producto puede aparecer en **varias transacciones**.) Siendo así he tenido que utilizar una PK compuesta de 2 variables: product_id + transaction_id, porque esta combinación sí que es única.

```sql
-- Nivel 3 - Exercici 2

-- Creación de la tabla intermedia venta_producto
CREATE TABLE venta_producto (
    transaction_id VARCHAR(50),
    product_id VARCHAR(50),
    PRIMARY KEY (transaction_id, product_id),
    FOREIGN KEY (transaction_id) REFERENCES transactions(id),
    FOREIGN KEY (product_id) REFERENCES products(id)
);
```

```sql
-- Para rellenar la nueva tabla:
USE db_ex_8;
INSERT INTO venta_producto (transaction_id, product_id)
SELECT t.id, jt.product_id
FROM transactions t
JOIN JSON_TABLE(
    CONCAT('["', REPLACE(REPLACE(t.product_ids, ' ', ''), ',', '","'), '"]'),
    '$[*]' COLUMNS (product_id VARCHAR(50) PATH '$')
) AS jt;
```

Output

```
OK, 253388 rows affected in 4.545641899108887s
```

```
382    -- Comprobando valores de la nueva tabla
383    SELECT COUNT(*) FROM venta_producto;
```

Output | Result #1

COUNT(*)
253388

OK, 1 record retrieved in 62.832ms

Consulta para verificar que todos los productos que están en la tabla transaction también aparecen en la nueva tabla.

```
382
383     -- Comprobando que todos los productos que están en transaction están en la nueva tabla:
384     SELECT t.id,
385         LENGTH(t.product_ids) - LENGTH(REPLACE(t.product_ids, ',', '')) + 1 AS esperados,
386         COUNT(vp.product_id) AS insertados
387     FROM transactions t
388     LEFT JOIN venta_producto vp ON t.id = vp.transaction_id
389     GROUP BY t.id, t.product_ids
390     HAVING esperados <> insertados;
```

Output | Result #1

*

OK, 0 records retrieved in 1.4651129245758057s

Por fin la consulta que es solicitada:

```
393  -- Veces producto vendido
394  SELECT p.id, p.product_name, COUNT(vp.transaction_id) AS veces_vendido
395  FROM products p
396  LEFT JOIN venta_producto vp ON p.id = vp.product_id
397  GROUP BY p.id, p.product_name
398  ORDER BY veces_vendido DESC;
```

Output | Result #1

id	product_name	veces_vendido
52	riverlands the duel	2654
29	Tully maester Tarly	2635
21	duel Direwolf	2609
16	the duel warden	2608
66	mustafar jinn	2600
48	rock Renly in	2597
87	sith Jade	2597
33	duel warden	2597
23	riverlands north	2593
68	Stark Karstark	2589
88	Stannis warden south	2587
28	chewbacca mustafar	2584
73	Dorne bastard	2584
4	warden south duel	2584

OK, 100 records retrieved in 226.351ms